UE24CS352A · MACHINE LEARNING · PROBLEM #27

Predicting Bridge Damage under Earthquakes

Logistic Regression, QDA and KNN on 249 bridge and earthquake records

Guru R Bharadwaj PES1UG24CS177
Chukkapalli Rohan PES1UG24CS135
Section C, Team 3



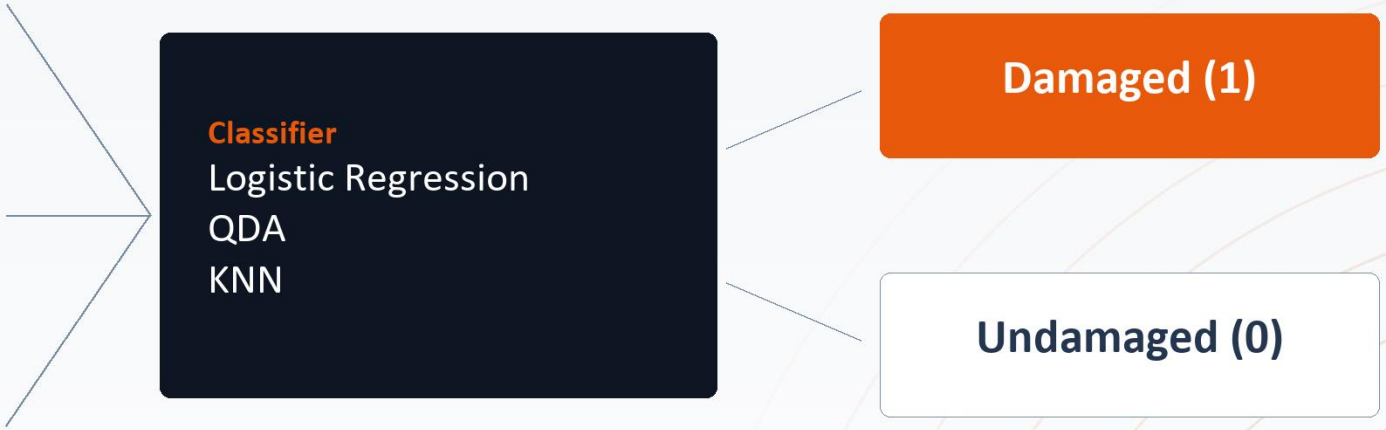
The problem

Based on Jiang and Zhang, Stanford CS229 (2018)

Bridge age
years since it was built

Earthquake magnitude
Richter scale

Distance to epicentre
in miles



Our additions: material and bridge type as extra features, and a second model that predicts the damage level (none, moderate, collapse).

The data

249 bridge and earthquake pairs from the authors' ds2.csv

249

records

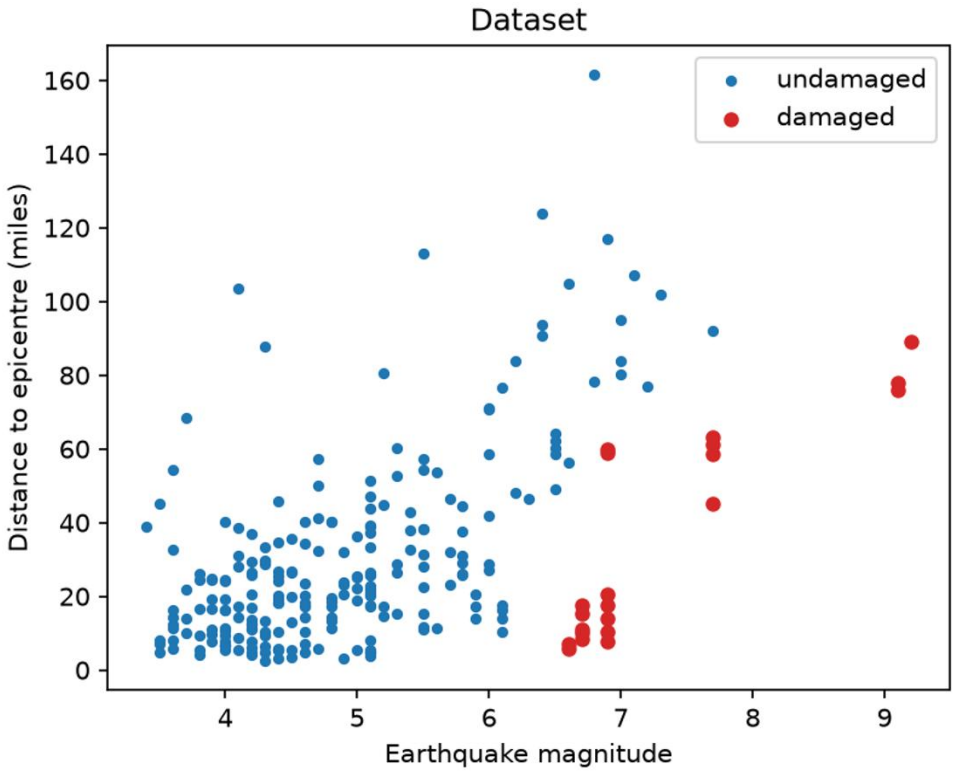
21

damaged (8.4%)

7

earthquakes behind every damaged row

- Damaged rows come from government reports on famous earthquakes
- Undamaged rows are other nearby USGS earthquakes, assuming no report means no damage
- We checked our cleaned file against the original, row by row



Damaged bridges cluster at high magnitude and short distance

How we evaluated

The same pipeline for every model, repeated over 100 random splits

1

Split first

Stratified 70/30, so the test set keeps the real 1:10 ratio

2

Up-sample train only

Bootstrap the damaged rows to ratios 0.2, 0.5 and 1.0

3

Fit three models

Scaled LogReg and KNN, QDA with reg_param 0.1, k picked by 5 fold CV on F1

4

Score on the test set

Recall and precision on damaged, F1, balanced accuracy, ROC AUC

92%

accuracy from always saying "undamaged". So we don't rank models on accuracy.

Extra test

Split by earthquake, so no event is in both train and test

Code structure

`src/data.py`

load, split, bootstrap, group by earthquake

`src/models.py`

three models, choose k by CV

`train_evaluate.py`

binary experiment, core or all features

`train_multiclass.py`

damage level 1, 2 or 3

`grouped_split.py`

test on unseen earthquakes

`predict.py`

command line demo using the saved best model

`results/`

every table and plot in these slides

`data/bridge_data.csv`

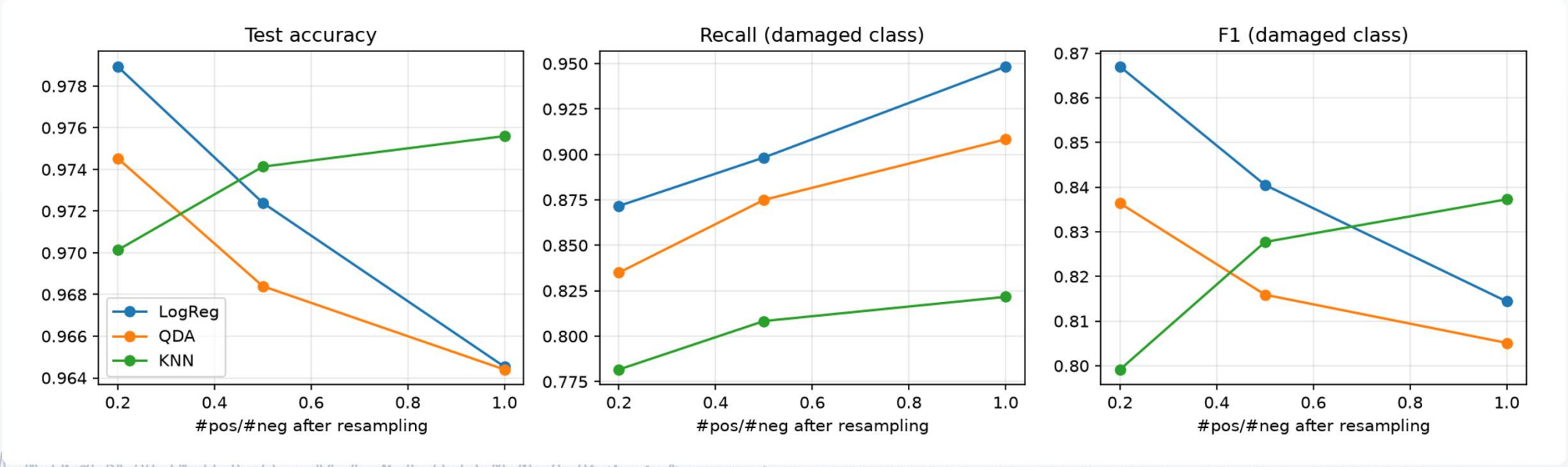
the 249 cleaned records

Flow: data.py → models.py → train_evaluate.py → models/*.joblib → predict.py

Results on random splits

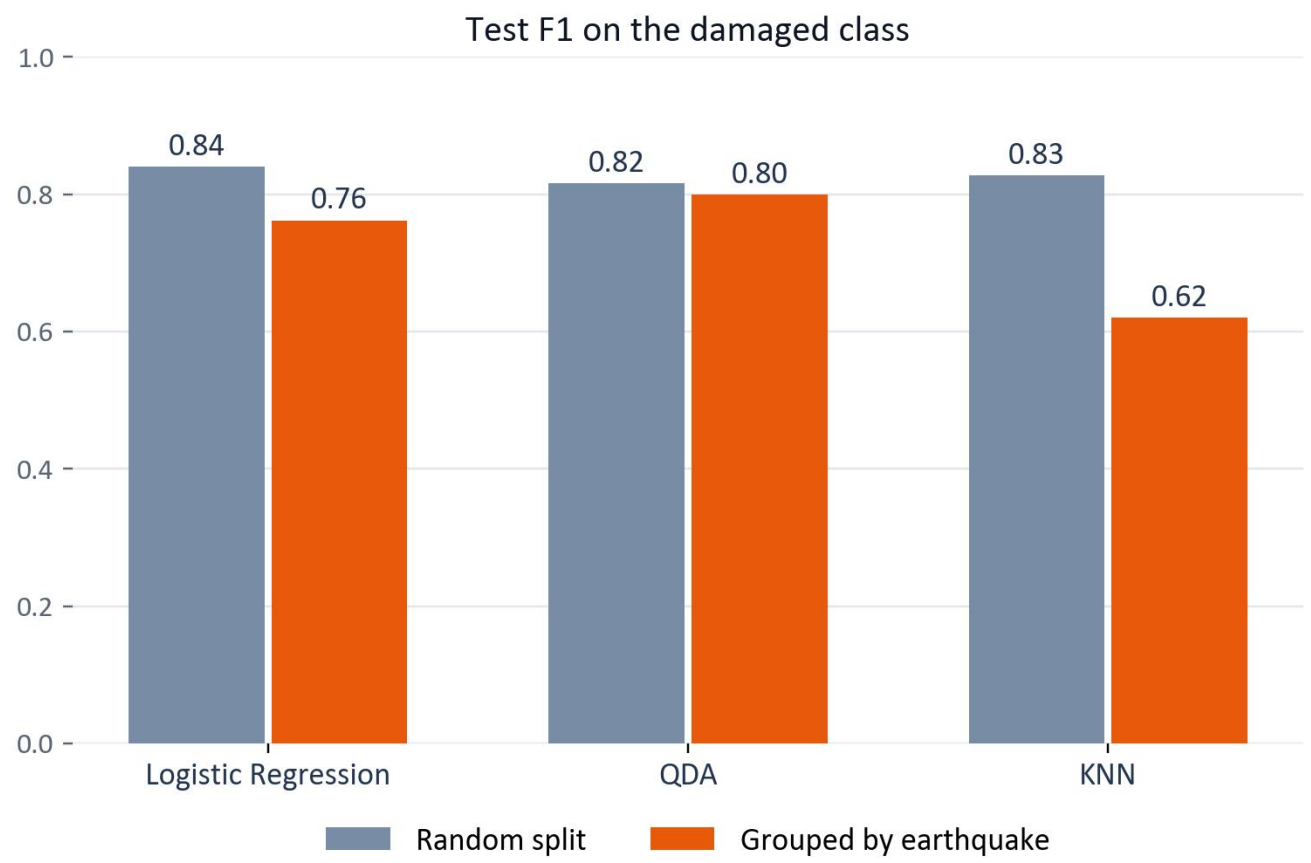
Core features, mean of 100 runs over three ratios. Best value in each column in orange.

Model	Test acc	Balanced acc	Recall	Precision	F1	AUC
Logistic Regression	0.972	**0.942**	**0.906**	0.803	**0.841**	**0.994**
QDA	0.969	0.925	0.873	0.793	0.819	0.992
KNN	**0.973**	0.896	0.804	**0.874**	0.821	0.906



Testing on unseen earthquakes

2 of the 7 damaging earthquakes held out per run, 100 runs, up-sampling ratio 0.5



KNN 0.83 → 0.62
It was partly matching rows from the same event

Logistic Regression 0.84 → 0.76
Keeps recall at 0.91, but raises more false alarms

QDA 0.82 → 0.80
Barely moves, and becomes the best model, as the original paper found

Extensions

Adding material and bridge type, and predicting the damage level

EXTRA FEATURES

0.841 → 0.849 Logistic Regression F1

0.821 → 0.672 KNN F1

DAMAGE LEVEL

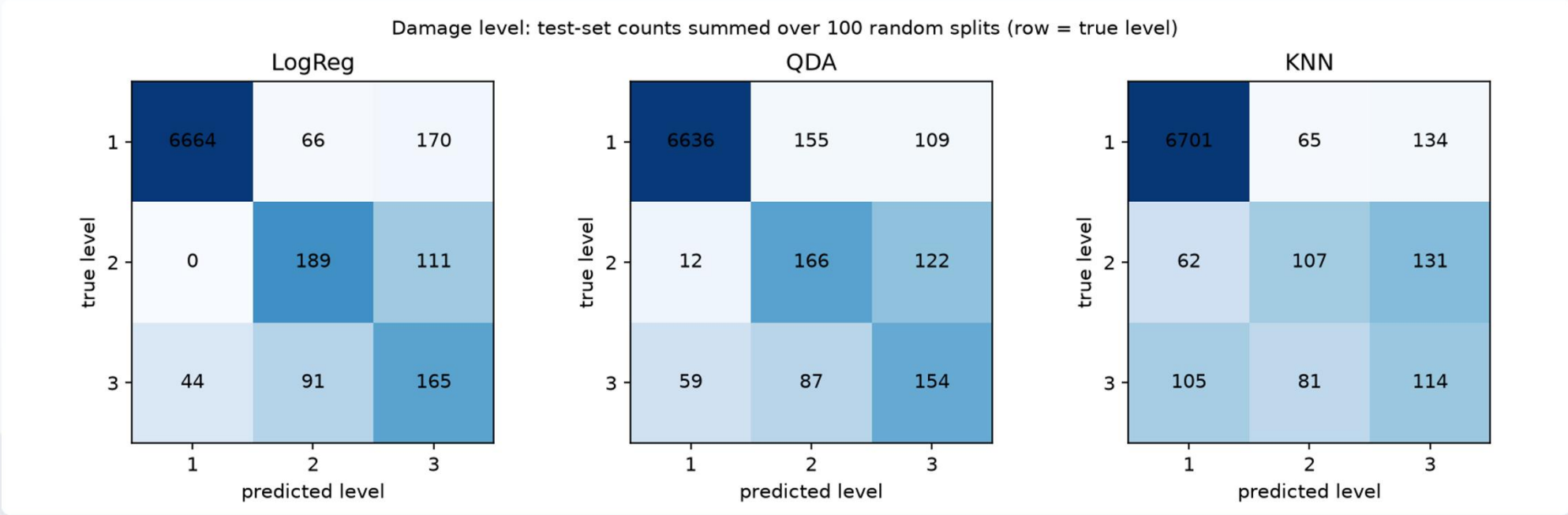
0.66

macro F1, Logistic Regression

SMALL CLASSES

11 and 10 moderate and collapse rows

0.36 to 0.63 recall on those levels



Live demo

PowerShell · ML Proj

```
PS> python predict.py --age 53 --mag 6.9 --dist 60
    Step 2: multiply by weights and add ...  total (logit)  0.505
    Step 3: sigmoid, 1 / (1 + e^-0.505) = 0.624
    P(damaged) = 62.4%  ->  DAMAGED

PS> python predict.py --age 53 --mag 6.9 --dist 60 --material steel --type beam_girder

PS> python predict.py --age 20 --mag 4.5 --dist 40
```



The first input is the Bay Bridge in Loma Prieta 1989. The script shows each step of the calculation and the three most similar records in the data.



Conclusions

1. Age, magnitude and distance separate the classes well in this dataset
2. Under a fair split by earthquake, QDA is the most dependable model
3. Scores are optimistic: 7 big earthquakes supply every damaged row, so magnitude does most of the work
4. Undamaged labels rest on missing reports, and bootstrap only repeats the same 21 positives
5. **More damaging events would help more than a different model**

